Experiment 10

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory

Hyperledger Fabric is a permissioned, enterprise blockchain framework: every participant has an identity issued by a certificate authority and belongs to an organization, unlike public chains where anyone can join. Its main components are:

- **Peers** — hold a copy of the ledger and execute chaincode to endorse transactions. The test network has `peer0.org1.example.com` (Org1MSP) and `peer0.org2.example.com` (Org2MSP).
- **Orderer** — orders endorsed transactions into blocks (`orderer.example.com`); it does not execute chaincode.
- **Channel** — a private ledger shared by chosen organizations; here `mychannel`.
- **Chaincode (smart contract)** — business logic installed on peers; here the `asset-transfer-basic` JavaScript chaincode, which manages assets with the fields ID, Color, Size, Owner and AppraisedValue.
- **Membership Service Provider (MSP)** and **TLS certificates** — identities and secured communication; the `CORE_PEER_*` variables select which identity the `peer` CLI acts as.

Fabric transactions follow *execute-order-validate*: a client sends a proposal to endorsing peers, which simulate it and sign the result; the client submits the endorsed transaction to the orderer, which puts it in a block; peers then validate and commit it. This is why `peer chaincode invoke` must name the orderer and the endorsing peers of **both** organizations (the chaincode's endorsement policy), whereas `peer chaincode query` is read-only and only talks to the local peer.

Tools used: Hyperledger Fabric 2.5.16 binaries and Docker images, `fabric-samples` `test-network` (2 peer organizations, 1 orderer, channel `mychannel`), the `asset-transfer-basic` JavaScript chaincode, Docker (OrbStack) on macOS. The manual's Windows Subsystem for Linux + Ubuntu environment was not used; the same commands were run natively on macOS through `run_fabric.sh`, with the elided `peer chaincode invoke ...` flags written out in full.

Output

1. Verify the Fabric binaries (manual step 6)

```
$ peer version
peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: darwin/arm64
```

The Fabric CLI is on the `PATH` with `FABRIC_CFG_PATH` set, and the Docker images for peer, orderer, CA, ccenv and baseos were pulled by `install-fabric.sh`.

2. Start the network and create the channel (step 7)

```
$ ./network.sh up createChannel
 Container peer0.org1.example.com Started
 Container orderer.example.com Started
 Container peer0.org2.example.com Started
Anchor peer set for org 'Org1MSP' on channel 'mychannel'
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
```

Two peers and an orderer start as Docker containers and both organizations join `mychannel` and set their anchor peers.

3. Deploy the chaincode (step 8)

```
$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript
Chaincode definition approved on peer0.org1 on channel 'mychannel'
Chaincode definition approved on peer0.org2 on channel 'mychannel'
Chaincode definition committed on channel 'mychannel'
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true,
Org2MSP: true]
```

The chaincode is packaged, installed on both peers, approved by both organizations, and committed to the channel.

4. Initialize the ledger and query (steps 10 to 12)

```
$ peer chaincode invoke ... -c '{"function":"InitLedger","Args":[]}'
Chaincode invoke successful. result: status:200

$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},
 {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},
 {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, ...]

$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

`InitLedger` is a transaction that writes six sample assets; `GetAllAssets` (read-only) lists them and `ReadAsset` fetches `asset5` by ID.

5. Create a new asset owned by Yash (step 13)

```
$ peer chaincode invoke ... -c '{"function":"CreateAsset","Args":["asset7","green","12","Yash","950"]}'
Chaincode invoke successful. result: status:200 payload:"
{\"ID\":\"asset7\",\"Color\":\"green\",\"Size\":12,\"Owner\":\"Yash\",\"AppraisedValue\":950}"

$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset7"]}'
{"AppraisedValue":950,"Color":"green","ID":"asset7","Owner":"Yash","Size":12}
```

A new asset, `asset7`, is created with owner Yash, and reading it back confirms the write was committed to the ledger.

6. Transfer the asset (step 14)

```
$ peer chaincode invoke ... -c '{"function":"TransferAsset","Args":["asset7","Tomoko"]}'
Chaincode invoke successful. result: status:200 payload:"Yash"

$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset7"]}'
{"AppraisedValue":950,"Color":"green","ID":"asset7","Owner":"Tomoko","Size":12}

$ peer chaincode query -C mychannel -n basic -c '{"Args":["AssetExists","asset99"]}'
false
```

`TransferAsset` returns the previous owner (“Yash”) and the next read shows the owner is now Tomoko. `AssetExists` confirms a non-existent ID returns `false`.

7. Stop the network (step 15)

```
$ ./network.sh down
 Container orderer.example.com Removed
 Container peer0.org2.example.com Removed
 Container peer0.org1.example.com Removed
```

All Fabric containers, volumes and generated chaincode images are removed.

(The complete run log, including every command and its output, is saved as `fabric_run_clean.log`, produced by `run_fabric.sh`, both in this folder.)

Conclusion

This experiment implemented a Hyperledger Fabric network (two peer organizations, one orderer and a channel, `mychannel`) and deployed the `asset-transfer-basic` chaincode with both organizations approving the definition. Asset management was demonstrated end to end: the ledger was initialized with six assets, queried (all assets and one by ID), a new asset `asset7` was created with owner Yash, and ownership was then transferred to Tomoko, with a read-back after each transaction confirming the committed state. Read operations (`query`) worked directly against one peer, while writes (`invoke`) had to be endorsed by both organizations and ordered into a block, illustrating Fabric's execute-order-validate model and the difference from the public, Proof-of-Work chains of the earlier experiments. The run was done natively on macOS rather than in WSL/Ubuntu as the manual assumes, and used the standard sample asset case study with the asset owned by the author, not a separately designed case.